Chapter 3

Coding with Chatbots

IN THIS CHAPTER

- **Engineering prompts**
- **Chatting with Copilot**
- **Talking code with ChatGPT**
- **Getting schooled on the OpenAI Playground**

Large language models (LLMs) enable computers to understand and generate human languages with astonishing accuracy. *Chatbots* are easy-to-use interfaces to LLMs that enable conversations with a generative AI model. Using a chatbot, anyone can converse with an LLM in a way that's similar to how you would message a friend or coworker. (To learn about how machine learning and LLMs work, check out Book 2.)

Because the generative AI models that underlie AI chatbots have been trained on an enormous amount of text, it can sometimes seem like the chatbots are omniscient. However, after some time working with chatbots and sometimes getting responses that are incoherent or plainly wrong, you'll realize that they're not.

In this chapter, you learn how to use several popular AI chatbots to generate programming code. You also learn how to get better results by improving the inputs, or prompts, that you give an LLM.

Improving Your Prompts

Prompt engineering is the art and science of developing and optimizing prompts to use LLMs more effectively. By learning about prompt engineering, you'll not only become better at getting what you want out of LLMs but also gain a greater appreciation for what LLMs can and can't do.

Adjusting the temperature

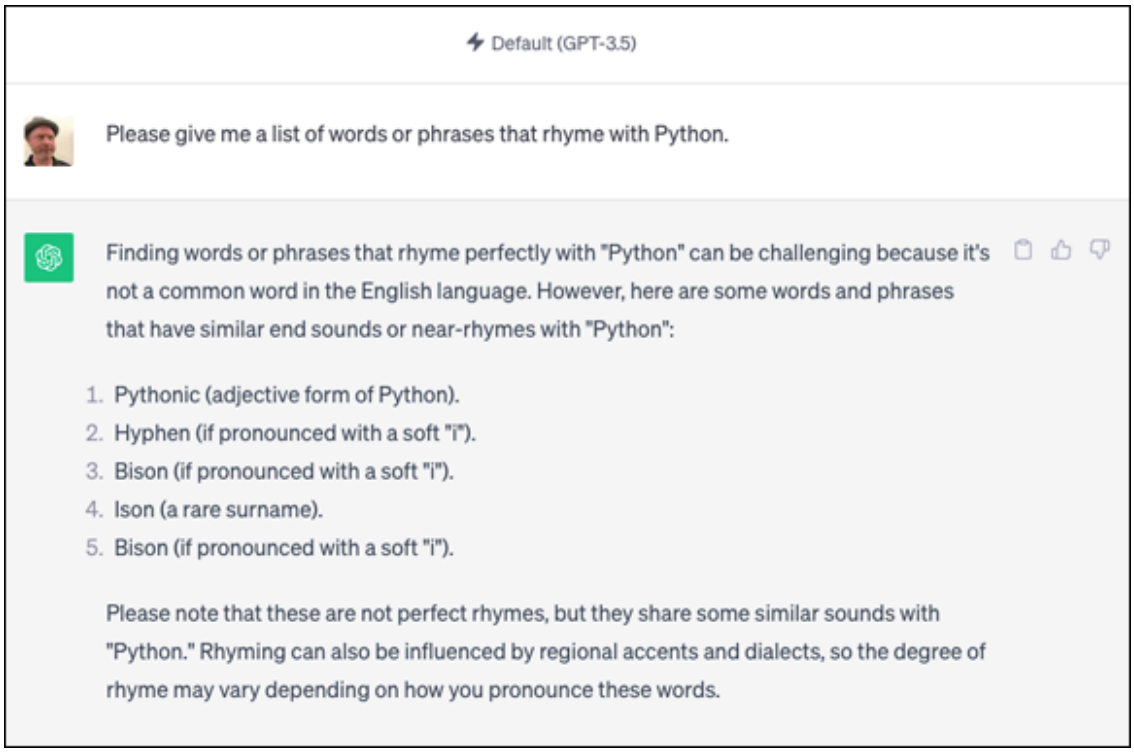
The results you get from an LLM are greatly influenced by a setting called *temperature,* which directly determines how creative or factual the model will be. When set to a lower temperature, the LLM will always return the most probable responses. When set to a higher temperature, the LLM will be more creative and random.

In most language models, the temperature scale goes from 0 to 1. For ChatGPT, the default setting is 0.7. Depending on the type of response you want, you may want to set this value lower or higher.



It's possible to set the temperature to as high as 2.0 with OpenAI's models, but temperature settings above 1.0 are more likely to be gibberish.

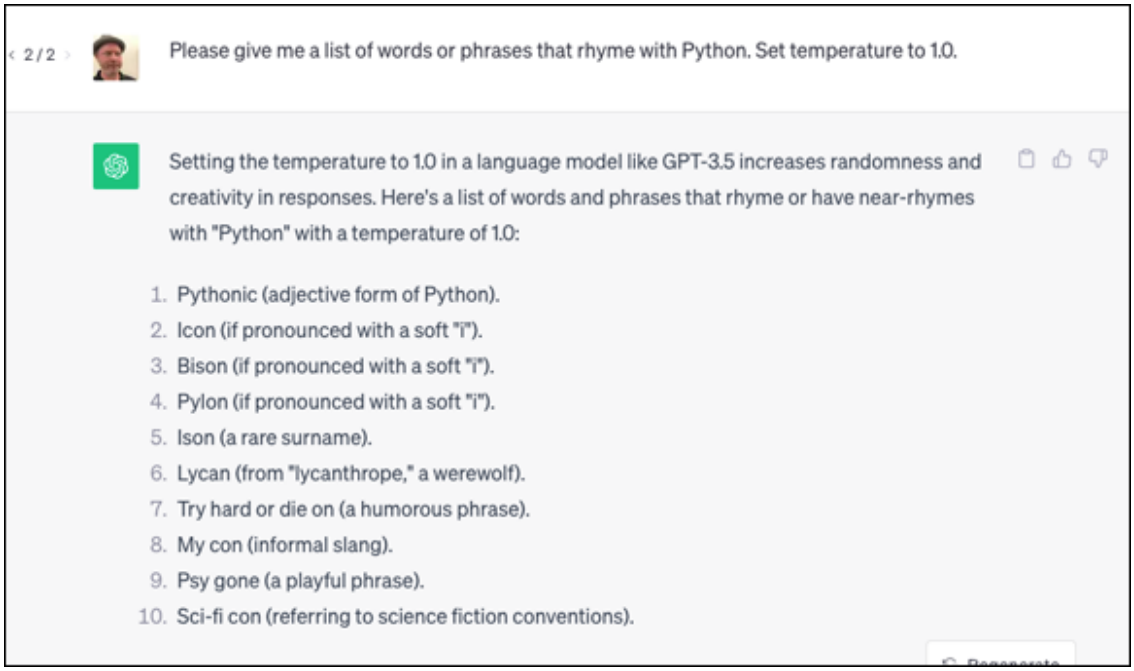
To illustrate what a difference the temperature makes, I asked ChatGPT for a list of words that rhyme with *Python.* I didn't specify a temperature, so ChatGPT used its default value. The response is shown in Figure 3-1. It told me that Python isn't a common word in English and gave me four words that all sort of rhyme with *Python* and repeated *bison* as the fifth word — presumably because it figured that four items weren't enough for a "list of words," but it couldn't come up with anything else.



ChatGPT

FIGURE 3-1: Asking ChatGPT to rhyme at 0.7 temperature.

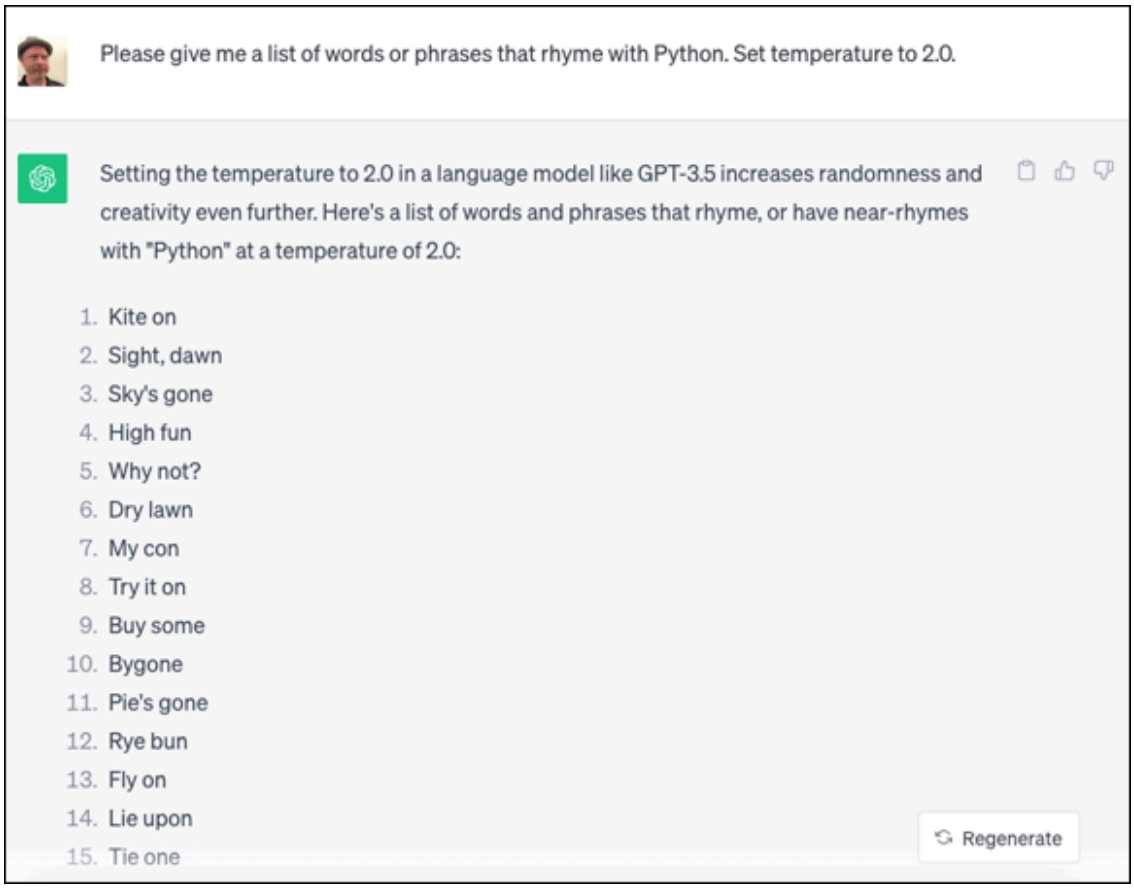
Next, I asked ChatGPT the same question but told it to set the temperature to 1.0. The response is shown in Figure 3-2.



ChatGPT

FIGURE 3-2: Asking ChatGPT to get more creative.

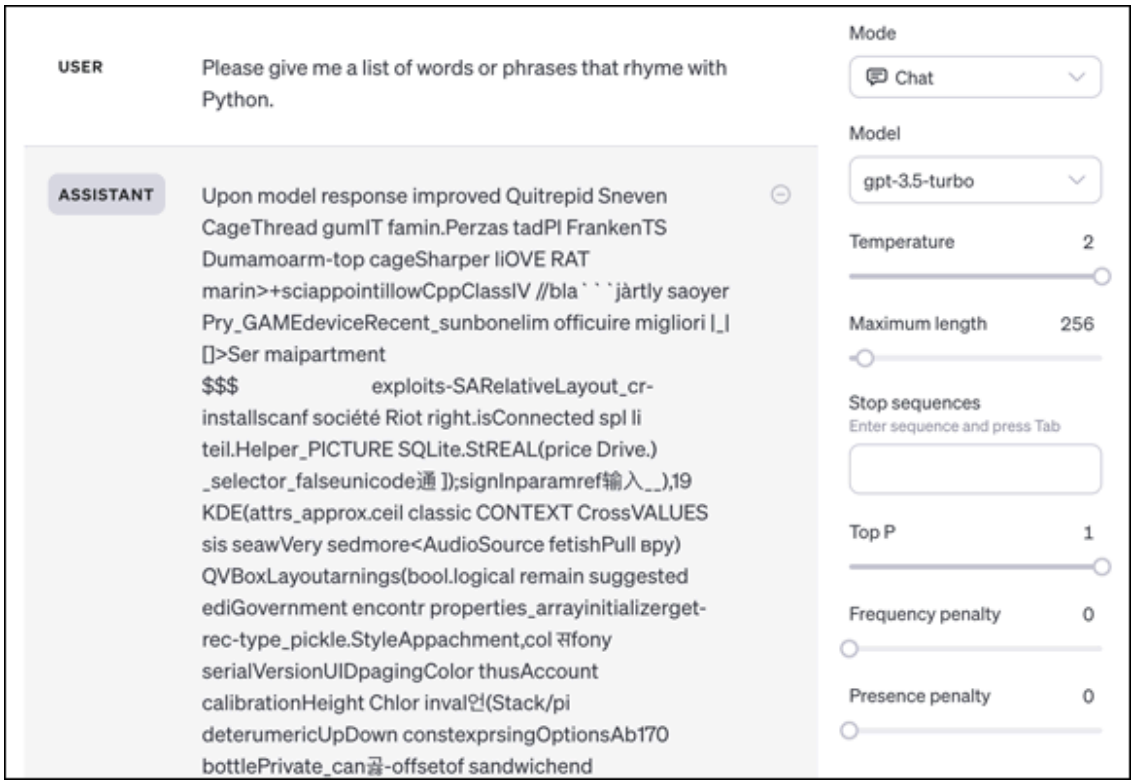
This time, it returned ten words or phrases, and eight of them were rhymes. Next, I asked ChatGPT to set the temperature to 2.0. The response is shown in Figure 3-3. It returned 19 suggestions for words that rhyme with Python, including my favorite suggestion: *Thai cuisine.*



ChatGPT

FIGURE 3-3: Setting the temperature to 2.0 in ChatGPT.

Because ChatGPT is tuned to always generate something sensible (even when it's being creative), adjusting the temperature doesn't have as great an effect on the output as adjusting it when working with OpenAI's models directly. To illustrate what it looks like when you crank up the heat on the model, Figure 3-4 shows the output for the same prompt with the temperature set to 2.0 but doing it through the API directly rather than through ChatGPT.



ChatGPT

FIGURE 3-4: What it looks like when the model gets too creative.

I cover how to work with the OpenAI models through the API later in this chapter.



WARNING Some AI assistants may allow you to set the temperature even higher than 2 if you ask nicely. However, be aware that higher temperatures will almost certainly generate garbage.

Deciphering the elements of a prompt

Prompts can be simple, such as the completion request *bread and,* or complex and multi-part. Prompts may include all or just some of the following elements:

- **Instruction:** A task you want the model to do. For example, *Translate the following text into Spanish.*
- **Context:** Additional information or files that the model should consider while generating the response. The files you have open in neighboring tabs while using Copilot are examples of context.

- **Input data:** The input or question you want the AI to answer or respond to. If your instruction is to translate some text to Spanish, the input data would be the text you want translated.
- **Output format:** The type or format in which the model should provide the output. Examples of output format are JSON data, tab-delimited data, and markdown format.

Open-ended versus closed-ended prompts

Prompts may be either open-ended or closed-ended. *Closed-ended prompts* ask for a specific and targeted response, such as

- Write a function to validate an email address input.
- Check this function for bugs.
- What is the tallest building in Indianapolis?

Open-ended prompts are designed to generate expansive responses, such as

- Write an email to my landlord telling him that I'm moving out if he doesn't fix the leak.
- Draft an outline for an article about nutrition and school lunches.
- Write a story about a magician squirrel in the style of Dr. Seuss.

Using different types of prompts

A chatbot will do its best to return an accurate completion for any prompt that you give it. However, you can use various types of prompts for different purposes, as you explore in this section.

Zero-shot prompting

In a *zero-shot prompt,* you don't give the model context or examples of what you're looking for, but instead rely entirely on its training. An example of a zero-shot prompt is

```
What is the capital of Maine?
```

For simple questions or tasks, many LLMs can do zero-shot prompting. As your requests become more complex, however, you'll need to move on to

other forms of prompts.



TIP As models have become larger, an increasing number of tasks can be accomplished using simple zero-shot prompts. However, if you want an LLM to do a task involving math or complex reasoning, you may have better success with one of the prompt types described in the following sections.

Few-shot prompting

In *few-shot prompting*, you start by explaining the parameters of a correct response and giving at least one example. Here's an example of a few-shot prompt to generate fake data for an API:

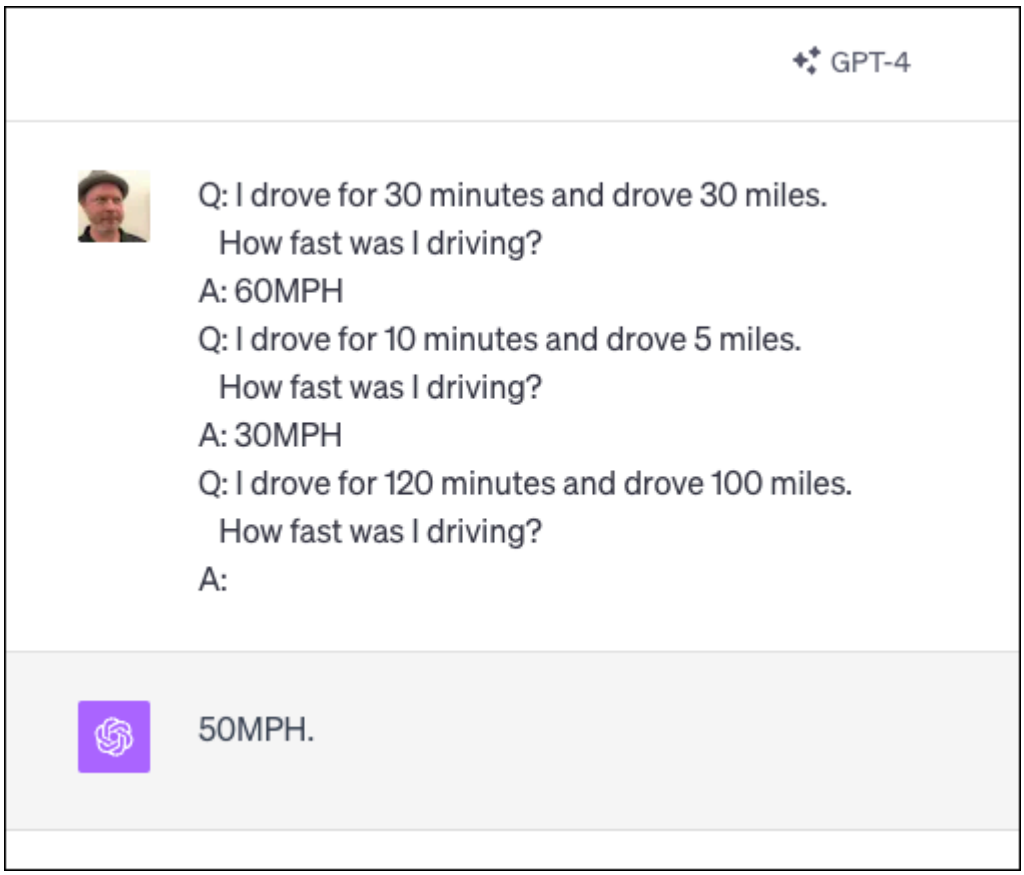
```
Give me 20 made-up records for customers,
in JSON format, with the following shape:
[
  {
    "Title": "Ms.",
    "GivenName": "Geneva",
    "MiddleInitial": "W",
    "Surname": "Cole",
    "StreetAddress": "3447 Reeves Street",
    "City": "Mill Center",
    "State": "WI",
    "ZipCode": 54301,
    "Birthday": "1/5/1978"
  },
```

A common technique in few-shot prompting is to provide examples of correct answers using Q&A format, as in this example:

```
Q: I drove for 30 minutes and drove 30 miles.
   How fast was I driving?
A: 60MPH
Q: I drove for 10 minutes and drove 5 miles.
```

```
   How fast was I driving?
A: 30MPH
Q: I drove for 120 minutes and drove 100 miles.
   How fast was I driving?
A:
```

The model will respond in the same format as in your examples, as shown in Figure 3-5.



ChatGPT

FIGURE 3-5: Few-shot programming gives examples and specifies the expected format.

Chain-of-thought prompting

In a *chain-of-thought prompt,* a series of intermediate reasoning steps is provided to improve the model's capability to perform complex tasks. For example, you could improve the accuracy and speed of the AI's response to the few-shot prompt in the preceding section by explaining how you came up with your answers:

```
Q: I drove for 30 minutes and drove 30 miles.
   How fast was I driving?
A: If you drove 30 miles in 30 minutes, that's 1 mile
   per minute (30 miles / 30 minutes), or 60MPH
   60MPH = (30 miles / 30 minutes) * (60 minutes / 1 hour)
Q: I drove for 10 minutes and drove 5 miles.
   How fast was I driving?
A: If you drove 5 miles in 10 minutes, that's.5 miles
   per minute (5 miles / 10 minutes), or 30MPH
   30MPH = (5 miles / 10 minutes) * (60 minutes / 1 hour)
Q: I drove for 120 minutes and drove 100 miles.
   How fast was I driving?
A:
```

Once again, the model will work through the problem in a step-by-step way that more or less follows the same format as the examples you provided. According to a 2022 study from Google Research ("Chain-of-Thought Prompting Elicits Reasoning in Large Language Models"), chain-of-thought prompting can dramatically improve an AI's capability to complete arithmetic, commonsense, and symbolic reasoning tasks accurately.

Prompting like a pro

Although prompting a language model often involves trial and error, you can improve your prompts by remembering the following tips:

- **Keep prompts short and precise.** Rather than stuffing every bit of information the model will need into one prompt, split complicated prompts into shorter ones that build on the previous ones in the conversation.
- **Use *continue*.** This tip is especially useful when generating large code blocks. A chat interface will limit the amount of output it gives for each prompt, so you may get only part of a function at once. Prompting with *continue* will cause the model to return additional content.
- **Use *Act as a* or *You are a*:** Follow the phrase with a description of the ideal personality or person for a task (such as *professional software developer, experienced database administrator,* or *helpful AI assistant who responds in pirate-speak*). This phrasing can result in higher-quality output.
- **Tell the model to follow current industry best practices.** This tip will lead to fewer usages of obsolete techniques or deprecated syntax.

- **Provide cues.** Indicate where and how the model should provide its completion when possible. For example, when asking a model to summarize an article, you might write the following to get the model to respond with a bulleted list:

```
The key points of this article are: *.
```

- **Label the prompt elements.** Specify the parts of your prompt that are instruction, input, context, or an output format. For example:

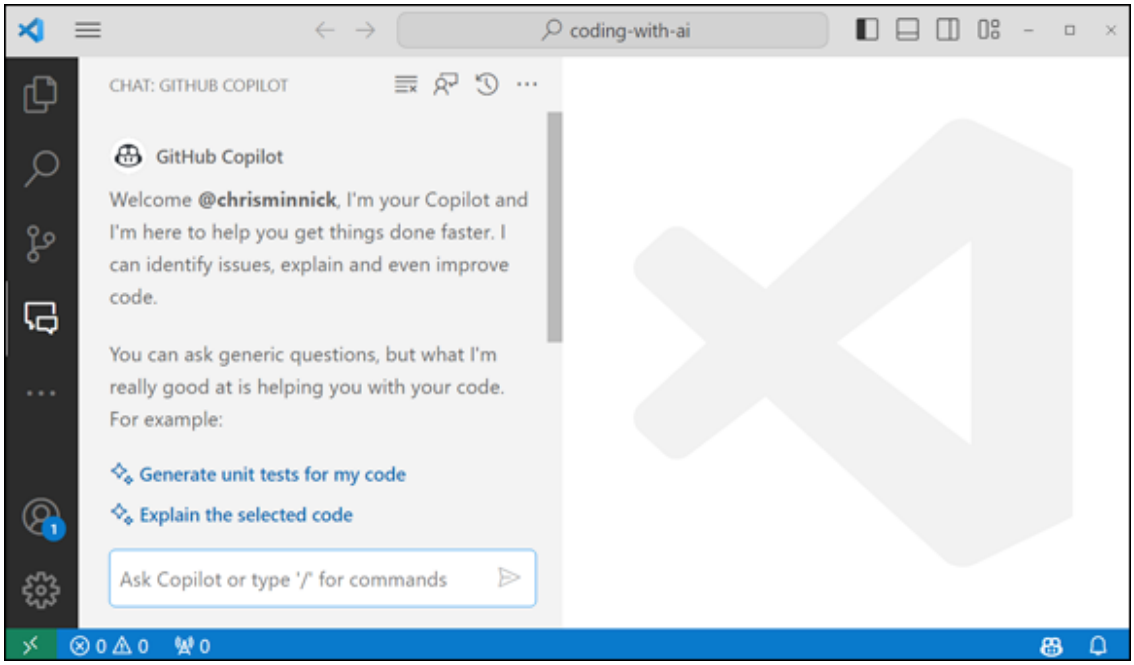
```
Input: text of an email.
Instruction: Write a response to this email with a list of action items.
Output format: Professional email with a bulleted list of next steps I'll take in response to the email.
Context: The sender is my boss.
```

- **Evaluate the output and ask for improvements if needed.** Always remember that you're the expert, and never simply accept a response you're not sure about.

Chatting with Github Copilot

Copilot provides a direct interface to prompting its model through the Chat extension. The Chat extension is installed automatically when you install Copilot, and you can access it by clicking the Chat icon in the left panel of VS Code.

The first time you access Chat, you'll see some basic instructions for using it, as shown in Figure 3-6.



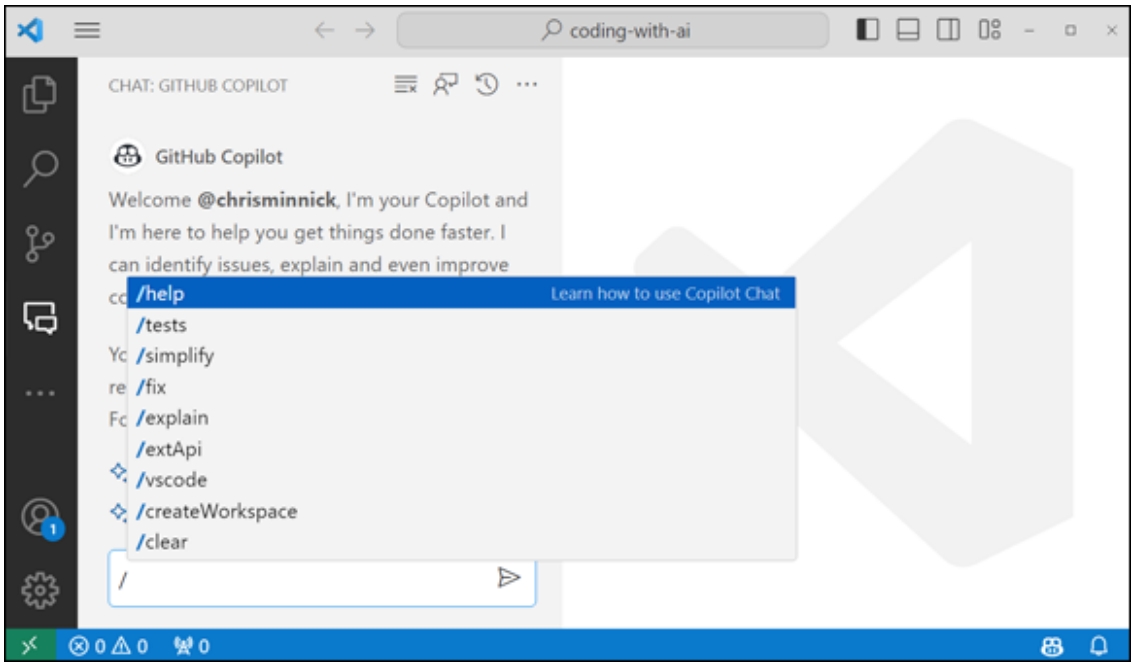
Microsoft Corporation

FIGURE 3-6: The Copilot Chat plug-in.

Understanding slash commands

Copilot chat has several built-in *slash commands,* which are shortcuts you can use to accomplish certain tasks or find out something. To see a list of slash commands, enter a forward slash character (/) into the chat input text box, as shown in Figure 3-7.

To get more information about the slash commands and other things you can do with Copilot Chat, type /**help** in the text box.



Microsoft Corporation

FIGURE 3-7: The slash commands in Copilot.

Knowing Copilot's agents

Agents are a feature of Copilot when it's used with VS Code. *Agents* allow you to further specify and expand the context that you want Copilot Chat to consider in its response. You invoke an agent by using @ followed by the name of the agent in your prompt. The following agents are available:

- **@workspace:** This agent makes Copilot Chat consider all the files in your current VS Code project when generating a response.

- **@vscode:** You can use this agent to ask questions specifically about how to use VS Code.
- **@terminal:** Use this agent to ask Copilot Chat how to do something in VS Code's integrated terminal.

The @workspace agent is a powerful tool. The best use for it is when you need Copilot to think holistically about your project, rather than just about the file or files you're currently viewing.

Getting the most out of Copilot Chat

Copilot chat works best when you treat it like it's a human coder — perhaps the most talkative human coder you've ever met but one who seems to know a lot, so you'll put up with it. For example, suppose you wanted help fixing a bug in your code. Your interaction with a more knowledgeable human colleague might go something like this:

- You: Hey, Linda, do you have time to help me fix something in my code?
- Linda: Sure. What's the problem?
- You: Do you see any potential security problems with the way I've written the signup and login endpoints in this server?
- Linda: Let me see. Nope. Looks pretty standard.
- You: How would you test it?
- Linda: Probably with Jest and Supertest.
- You: Yeah, okay. I don't know how to use those.
- Linda: Read the docs.
- You: Okay, thanks.

If you were to have the same conversation with Copilot Chat, your questions might be exactly the same (at least to start). Let's see how it goes. The following is my actual conversation with Copilot.

```
Me: Hey, Copilot, do you have time to help me fix some-
thing in my code?
Copilot Chat: Of course! I'd be happy to help. What seems
to be the issue?
Me: Do you see any potential security problems with the
way I've written the signup and login endpoints in this
server?
```

```
Copilot Chat: I'm not sure, could you show me the code for
those endpoints?
```

At this point, I opened the file containing the code I was referring to.

```
Me: Here they are.
Copilot Chat: I see that you're using JWTs to authenticate
users. That's a good start. However, I don't see any vali-
dation on the user input. You should always validate user
input to prevent attacks like SQL injection or cross-site
scripting (XSS). Additionally, it's important to hash
passwords before storing them in the database to prevent
them from being exposed in case of a data breach.
Me: I am validating the input, and I'm hashing the pass-
word using bcrypt. Can you look again?
Copilot Chat: I see that you're using the bcrypt library
to hash the password before storing it in the database,
which is great. However, I don't see any validation on the
user input. You should always validate user input to pre-
vent attacks like <!--<ce:anchor id="pp:581 np:582"
role="page-break"/>-->SQL injection or cross-site script-
ing (XSS). One way to do this is by using a library like
express-validator to validate the input before processing
it.
```

Here, Copilot was right. It turned out that I wasn't validating the input. Copilot also provided me with a suggested next prompt, as shown in Figure 3-8.



Microsoft Corporation

FIGURE 3-8: Copilot correctly identifies the problem with my code and offers to help.

I asked the follow-up question and implemented the suggested solution, but I still wasn't convinced. I asked the same question I had asked my co-worker Linda:

```
Me: How would you test it?
Copilot Chat: To test the validation, you can send a POST request to the /signup endpoint with invalid input and check if the server returns the correct error response. Here's an example of how you can test it using supertest and jest:
```

Not only did Copilot give me basically the same answer as my coworker, but it was much more patient and even wrote a sample test for me.

Chatting with ChatGPT

ChatGPT, which I introduced earlier in the chapter, stands for Chat Generative Pre-trained Transformer. This chatbot was developed by OpenAI to serve as a friendly front-end to the OpenAI GPT models. After its release on November 30, 2022, ChatGPT quickly became the fastest-growing consumer software application in history. ChatGPT converses with users and generates remarkably humanlike responses while considering context and previous prompts.

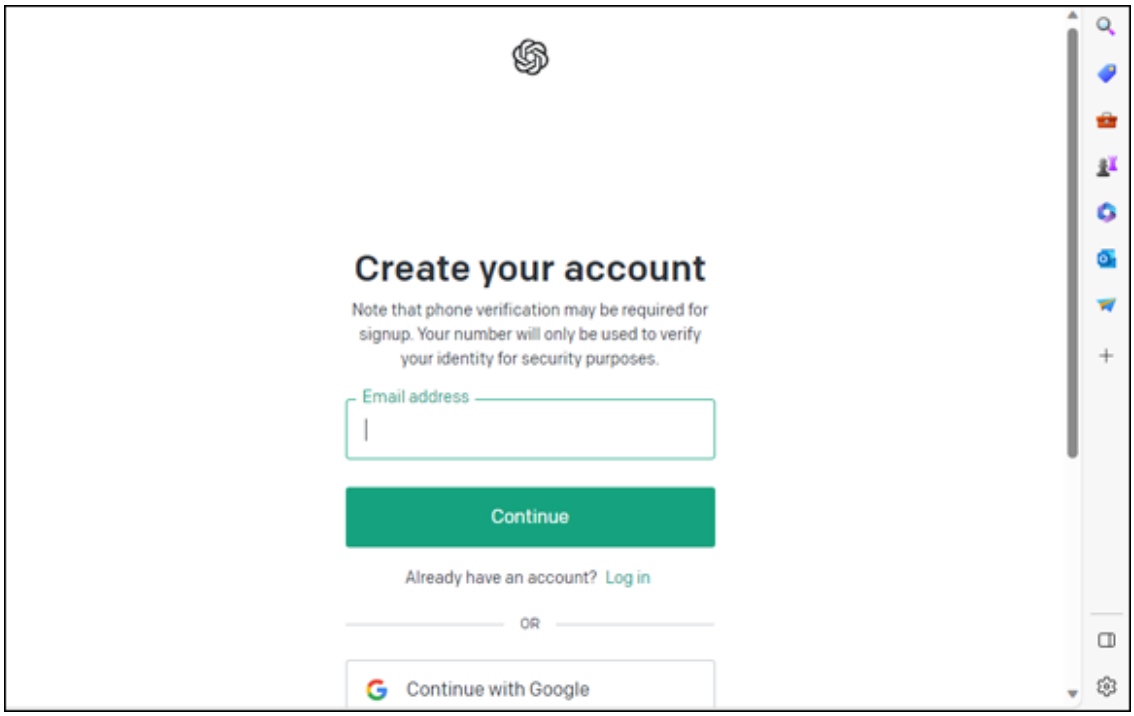
Unlike GitHub Copilot, ChatGPT isn't specifically trained to understand or generate code. Instead, it's tuned to be good at conversation. However, the enormous models it uses include programming code. Depending on the model you use with ChatGPT, it often handles prompts related to coding at least as well as Copilot.

ChatGPT is (currently) based on GPT-3.5 and GPT-4. The models have been further tuned through a process OpenAI calls reinforcement learning from human feedback (RLHF). Human trainers ranked responses from the model and created reward models to be able to converse with humans while using a safety system to avoid generating harmful content.

Because it's fine-tuned for conversation, ChatGPT can help with more creative tasks involving more than just a few lines of code. For example, I've used ChatGPT to generate sample data for an application I'm developing. I've used it also to help brainstorm ideas for new features to add to an app.

Signing up and setting up

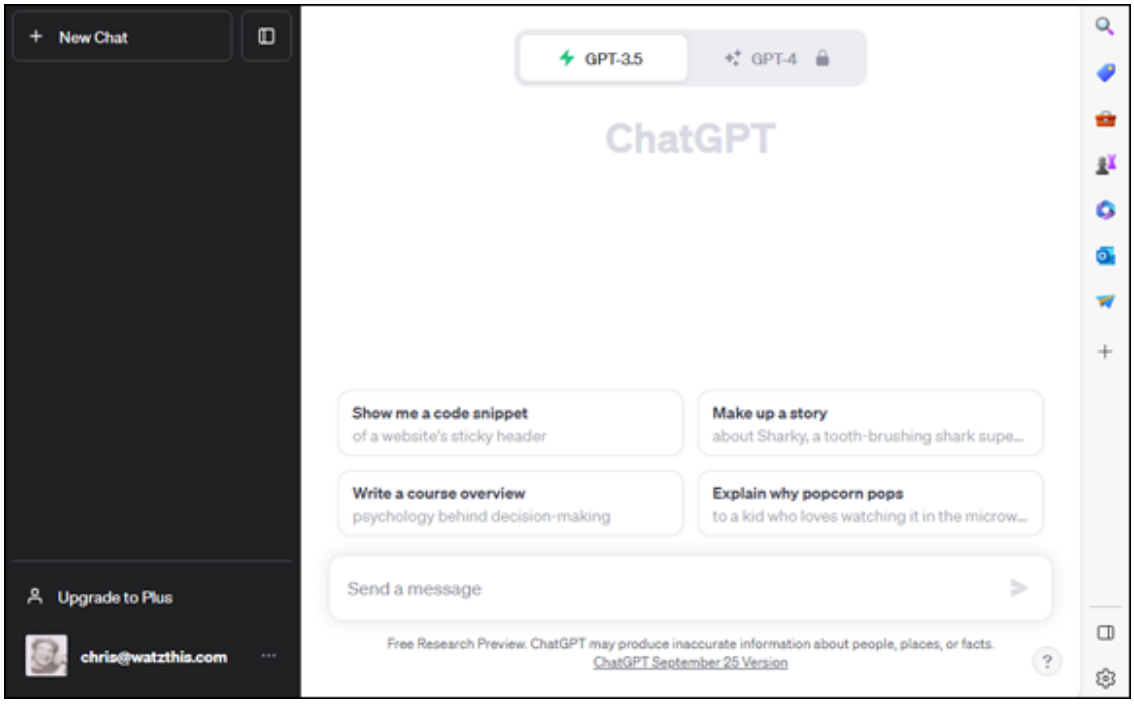
You can sign up for a ChatGPT account by going to `https://chat.openai.com`. Click the Sign Up link to get to the screen shown in Figure 3-9.



ChatGPT

FIGURE 3-9: Signing up for a ChatGPT account.

When you've finished the signup and phone number verification process, you'll see the ChatGPT user interface, as shown in Figure 3-10.



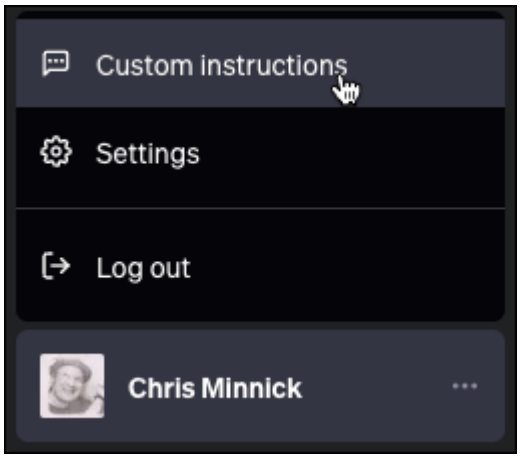
ChatGPT

FIGURE 3-10: The ChatGPT UI.

With a free account, you'll have access to an older language model, which is currently GPT-3.5. However, don't rush to sign up for a paid account just yet. The older model is faster than the newer one and is perfectly capable for our purposes.

Setting custom instructions

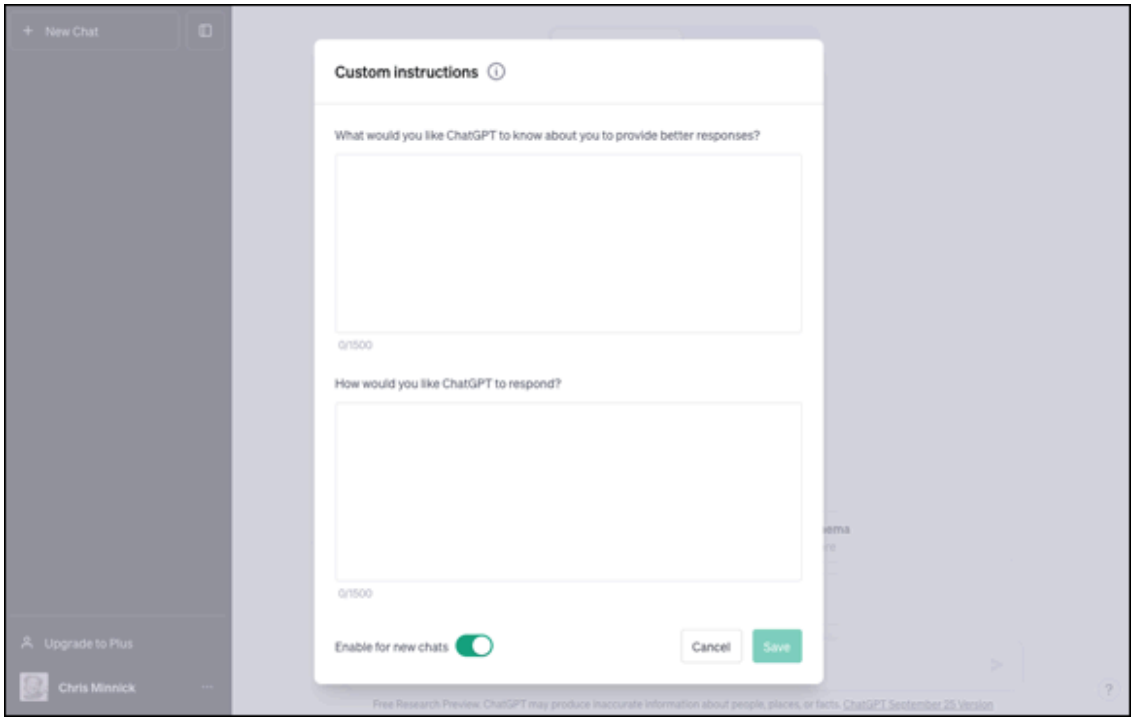
When you have a ChatGPT account, the first thing you'll want to do is to give ChatGPT some overall context about you and how you'd like the model to respond. You can do this by clicking your profile icon in the upper right of the ChatGPT interface and selecting Customize ChatGPT, as shown in Figure 3-11.



ChatGPT

FIGURE 3-11: Opening the Custom Instructions window.

In the Custom Instructions window that opens, which is shown in Figure 3-12, you can provide up to 1,500 characters about yourself and 1,500 characters describing how you'd like ChatGPT to behave and provide answers.



ChatGPT

FIGURE 3-12: The Custom Instruction window.



REMEMBER Earlier in this chapter, we discussed the four elements of a prompt: instruction, context, input, and output format. Here, you're providing some context and output format information.

Telling ChatGPT about you

The first box, labeled "What would you like ChatGPT to know about you to provide better responses?" is where you can provide context that will be applied to every conversation you have with ChatGPT. In the same way that you're able to have more meaningful interactions with your doctor, employee, or friends than you are with a stranger who knows nothing about you, this is the model's way of establishing some baseline understanding of what you may know already and who you are.

To see what ChatGPT suggests you include here, click the text area, and the help window shown in Figure 3-13 appears.

The tips provided are a great start. Don't worry about the structure or style of what you write. The important thing is to communicate any general information that the model should know to assist you better.



ChatGPT

FIGURE 3-13: Viewing ChatGPT's suggestions for the context custom instructions.

If you're using ChatGPT to help you with writing code, mention whether you're a student or a professional software developer, and include programming languages you know and your level of experience with each.

Figure 3-14 shows my current "about me" custom instruction.



ChatGPT

FIGURE 3-14: Example text for the first custom instruction.



TIP You don't have to use all 1,500 characters for either custom instruction. However, there's no reason not to.

Telling ChatGPT your expectations

In the second text area, you can give ChatGPT some general guidelines for how you'd like it to respond to your prompts. Just as with the first text area, if you click into the text area, you'll see a pop-up window that gives you some ideas about what to include in this section, as shown in Figure 3-15.



ChatGPT

FIGURE 3-15: Viewing ChatGPT's suggestions for the output format custom instructions.

To make ChatGPT more helpful as a coding assistant, you might want to provide the following instructions in this text area:

- You are an experienced professional software developer.
- Provide accurate responses to coding questions and flag uncertainties.
- Since I'm an expert in AI, skip explanations about your limitations.
- Examples of programming code should use the latest syntax and best practices.
- When I ask for programming code, I always just respond with the code and no explanations unless I specifically ask you to explain it.

You can modify your custom instructions any time you want, and you should do so whenever you have a new idea for how ChatGPT might be more helpful or pleasant to work with. Here are some additional ideas for things you may want to try adding to the second custom instruction:

- I always appreciate new ideas for ways to ask questions or form prompts that will help you be a better assistant to me.
- I like to be complimented. Sometimes, you should offer enthusiastic praise for a prompt or idea I give you.
- If I start my prompt with Pirate, you should respond in pirate-speak.

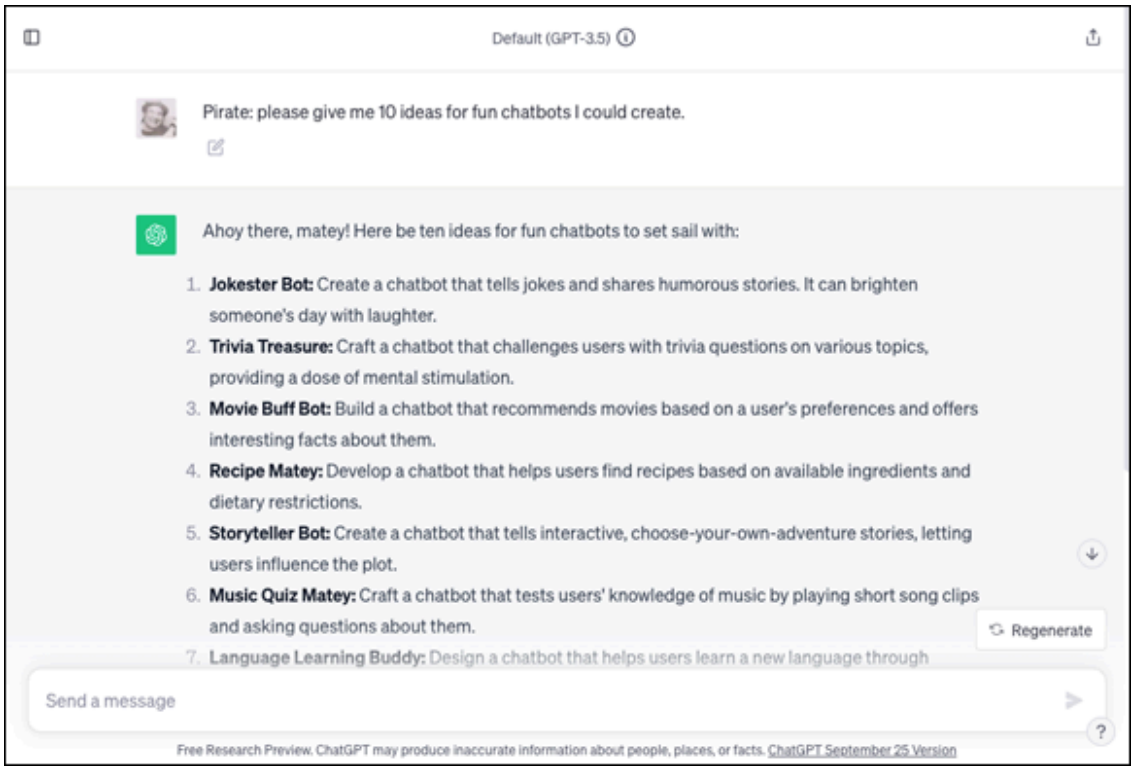
Below the two text input boxes is the Enable for New Chats switch, which is enabled by default. If you want to turn off your custom instructions temporarily (without deleting them), turn off this switch.

When you've finished editing your custom instructions, click the Save button.



TIP New instructions will apply only to new conversations, so if you do modify your instructions and want to try them out, remember to start a new chat.

Figure 3-16 shows the beginning of a new conversation that takes into account my new custom instructions.



ChatGPT

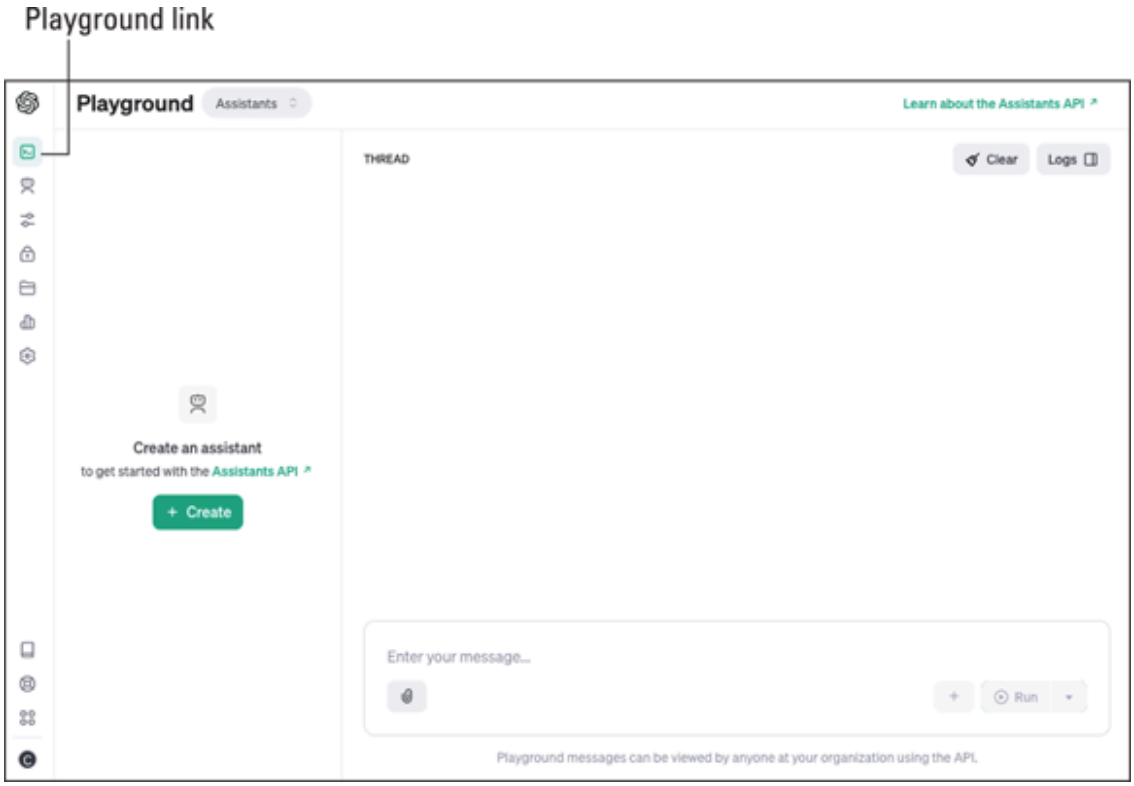
FIGURE 3-16: Custom instructions apply to all new chats.

Diving into the OpenAI Platform

Now that you're an expert at using ChatGPT, it's time to dive deeper into using the OpenAI models. In this section, you learn how to access OpenAI's developer platform, get an OpenAI API key, and build an application that makes use of the OpenAI API.

You can get more direct access to the OpenAI models by accessing the OpenAI developer platform site. Follow these steps to get started:

1. **Go to `https://platform.openai.com` in your web browser.**
2. **If you already have an account with OpenAI (which you do, if you've been using ChatGPT), click the Login button in the upper-right corner and log in.** Otherwise, sign up for a new account.
3. **Click the Playground link in the left navigation strip.**
   The icon for the Playground is the top one. The Playground interface opens, as shown in Figure 3-17.



ChatGPT

FIGURE 3-17: The OpenAI Playground.

Checking your credits

Using the OpenAI API and the Playground requires you to set up a payment method separate from your subscription to ChatGPT and to purchase credits. You can view the current rates for using different models by going to `https://openai.com/api/pricing/`. Pricing is based on the number of tokens of input you send to the model and the number of tokens the model outputs. For example, the GPT-4o model currently costs $2.50 per 1,000,000 tokens of input (about 750,000 words) and $10.00 per 1,000,000 tokens of output.



WARNING While you're experimenting with the OpenAI API, you're unlikely to accrue more than a few dollars in charges. If you build an application with the OpenAI API that becomes popular, you'll want to watch out that your costs don't get out of control.

If you just signed up for an OpenAI account, you may have been given free trial credits. To check whether you have credits in your account, go

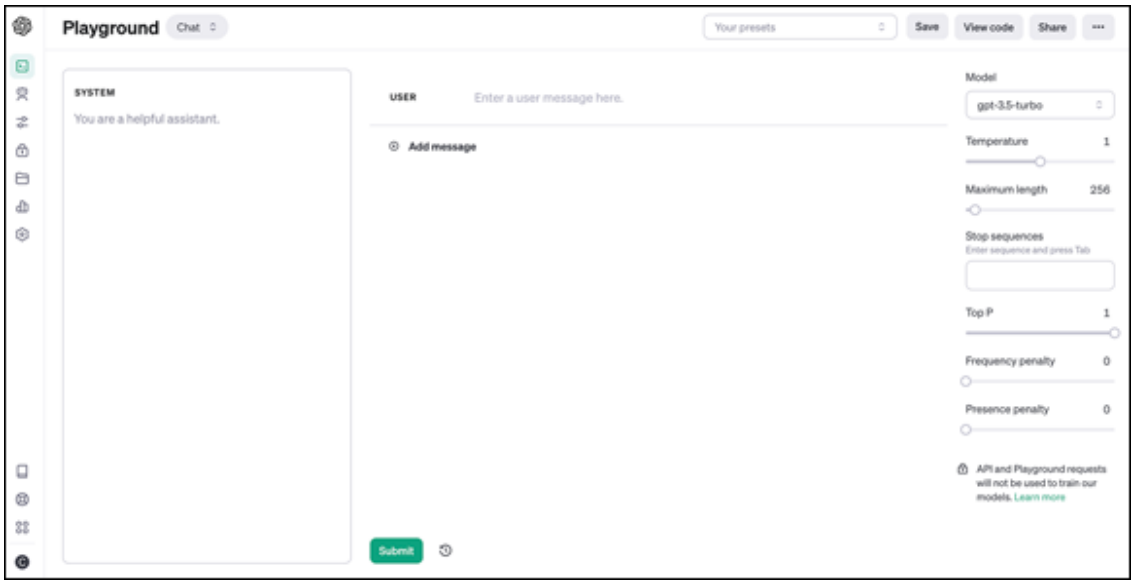
to `https://platform.openai.com/account/billing/overview`. If you don't have a free trial or if you have used all your free trial credits, you'll need to create a paid account, which you can do from this page as well.

After you have some credit in your account or you've set up a paid account, you're ready to use the OpenAI models in the Playground.

Messing around in the Playground

The OpenAI Playground (`https://platform.openai.com/playground`) gives you an interface for experimenting with the OpenAI API. The first time you access the Playground (refer to Figure 3-17), you'll see the Assistants interface, which provides an easy way to create an AI assistant.

To see how to customize the settings of a GPT model, you need to be in Chat mode. Select Chat from the left menu (see Figure 3-18).



ChatGPT

FIGURE 3-18: Chat mode in the OpenAI Playground.

Before you use Chat mode, however, you should know the following:

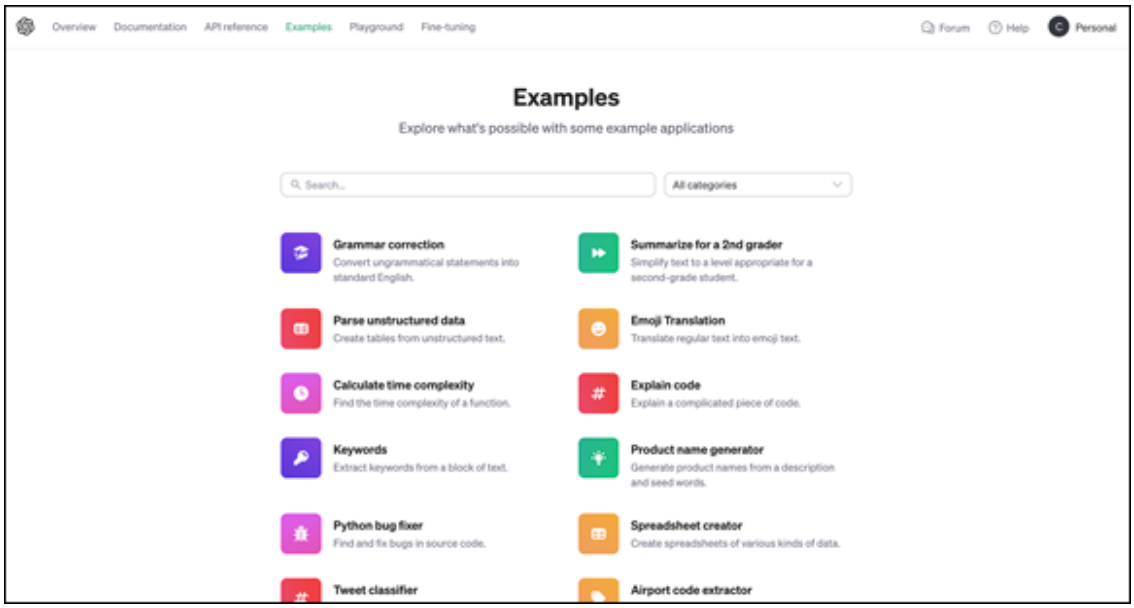
- You can enter instructions or choose a preset to get completions from the model.
- You can change the model to which your request will be sent.

- Use good judgment when sharing completions; you're free to attribute them to yourself or your company.
- Requests you send to the API aren't used to train the models.
- Currently, the cutoff for the default model is April 2023.

There isn't much obvious onscreen help for using the Playground. However, if you hover your cursor over the labels for the settings on the right, you'll see information about each setting. Additional help for both the Playground and the OpenAI platform, in general, is available through the documentation and help links in the left navigation strip.

Running examples

OpenAI provides examples of prompts to try, which you can access by clicking the Your Presets drop-down menu at the top right of the page (refer to Figure 3-18) and selecting Browse Examples. The Examples page, shown in Figure 3-19, displays samples of prompts. You can also search for prompts by using the Search box and the category drop-down menu.



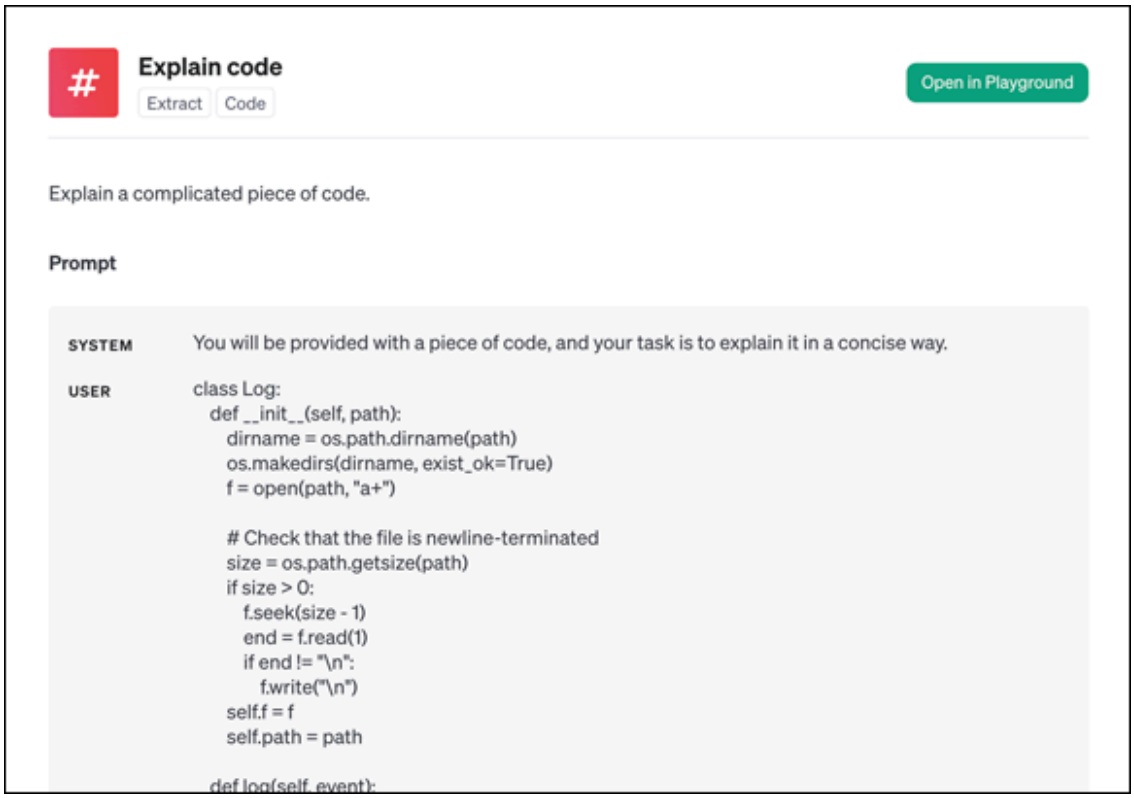
ChatGPT

FIGURE 3-19: The OpenAI Examples page.

When you click one of the example prompt descriptions, a pop-up window will appear that contains the instruction (which the Playground calls *system*) and sample input (which the Playground calls *user*), as shown in

Figure 3-20. Below the sample input, you'll see sample output from the application.

After you've looked through the sample input and output, click the green Open in Playground button at the top of the pop-up window to open the prompt in the Playground. The system and user parts of the prompts will be filled out for you, and the settings on the right side of the screen will also be preset to good values for the task at hand. If you open a coding-related example, note that the temperature setting will be set very low so that the model will return the most accurate response it can. If you open an example that requires creativity, such as the Product Name Generator example, the temperature will be set to a higher level.



ChatGPT

FIGURE 3-20: Viewing one of OpenAI's example prompts.

Playing the roles

OpenAI Playground's Chat mode has two large areas where you can input data to prompt the model. Within these two areas, you can enter text to play three different characters: system, user, and assistant.

The left text area is the system text area. Use this area to specify who or what you'd like the model to act like, and what the model should do with input that will be forthcoming (in the other text area). The default system message is *You are a helpful assistant.*

The text area to the right of the system input box is the user or assistant area. You can switch between the user and assistant roles by toggling the User or Assistant label in the message input field. Anything you label as user input will be given to the model after the system input. The assistant role is used by the model to respond to messages from the user role. You can also act as the assistant to give the AI model examples of what its output should look like.

Adjusting the model's settings

On the right of the Playground's Chat mode interface, choose the model you want to use. The default model is currently gpt-4o, but you can select any model that's available.



REMEMBER Some models cost more than others to use. You can check the pricing for OpenAI's different models at `https://openai.com/api/pricing`.

Below the model selection drop-down menu, you have access to the following additional settings:

- **Temperature** controls the randomness of responses.
- **Maximum Length** sets the maximum number of tokens that will be used when you submit your prompt. The tokens are shared between your prompt and the model's response.
- **Stop sequences** are combinations of characters that will cause the model to stop generating content. For example, if you want the model to return a numbered list containing ten items, you could set the stop sequence to 11, or if you want the model to return only a single line of text, you can set a carriage return as a stop sequence.
- **Top P** is another way to control the creativity and diversity of responses. Top-P can be set to a value between 0 and 1. (The *P* in *Top P* stands for *probability.*)

With a low Top P setting, the model considers only the most probable responses and will tend to generate predictable responses. The higher the Top P, the larger the pool of possible responses that the model will randomly select from when generating responses and the more diverse and creative the output will be.

- **Frequency penalty** determines how much new tokens will be penalized based on how many times they appear in the previously generated tokens. The value of the frequency penalty can be set to a value between 0 and 2. A higher frequency penalty will cause the model to generate more unique words.
- **Presence penalty** determines how much to penalize tokens based on their appearance in the previous text. The presence penalty can be set to a value between 0 and 2. A higher presence penalty will make the model have more diverse ideas.

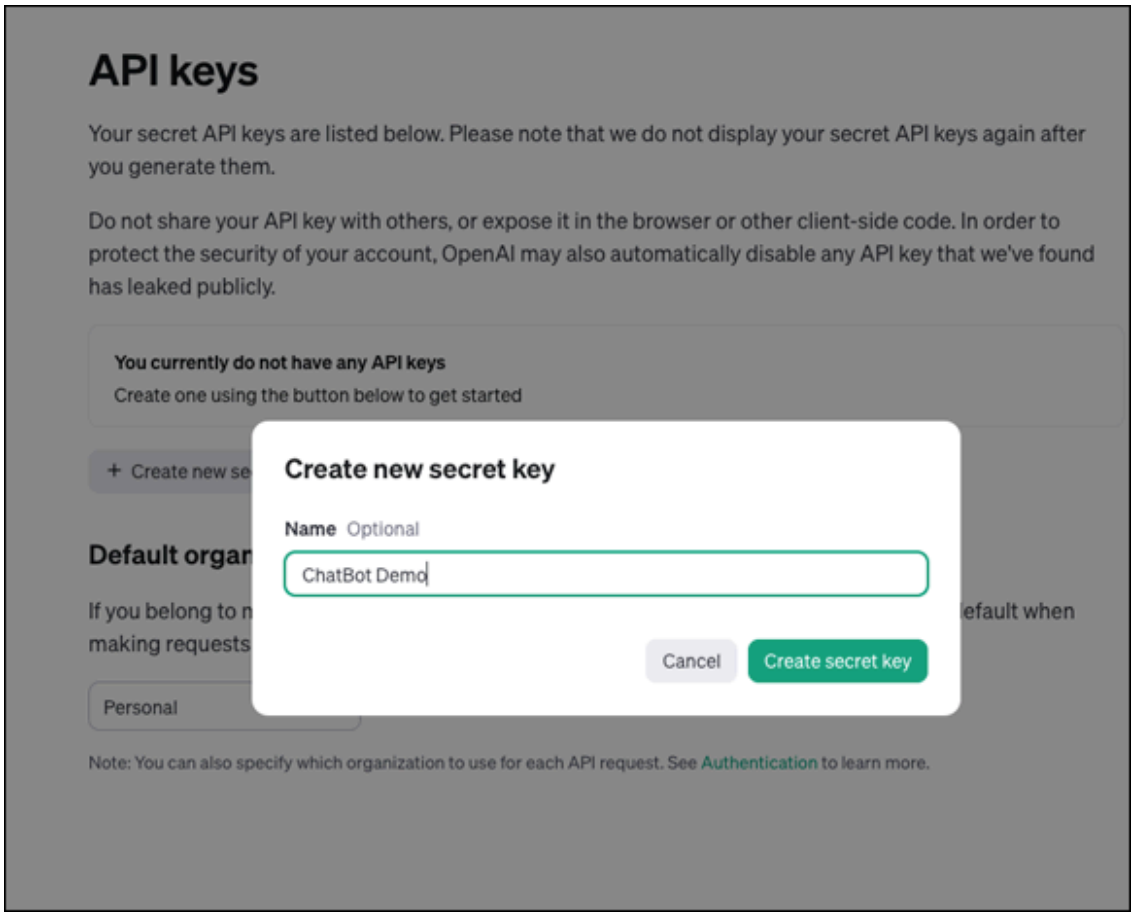


TIP When generating code, the frequency penalty and the presence penalty should both be set to 0 or a low value. You should set the frequency penalty to 0 because it's common and necessary for the same keywords (such as `def` in Python or `function` in JavaScript) to appear many times in a program. The presence penalty should be a low value to indicate that you prefer accuracy and consistency over seeing multiple ways of doing the same thing.

Getting an API key

Before you can write your own programs that make use of the OpenAI models, you have to have an API key. Follow these steps to get your key:

1. **Log in to the OpenAI Playground at `https://platform.openai.com/playground`.**
2. **Click the Gear icon in the top toolbar and then click API keys from the left menu.**
3. **Click the Create New Secret Key button and give your key a name that relates to what you're going to use it for, as shown in Figure 3-21.**
4. **Click Create Secret Key.**
   Your new API key appears.
5. **Copy your API key and save it where you won't lose it.**
   This is the only time OpenAI will show you your new secret key, so make sure not to rush through this step. However, if you do lose your secret key, it's easy enough to generate a new one.



ChatGPT

FIGURE 3-21: Naming your secret key.



WARNING Don’t commit any files containing your API key into your code repository.

Developing a Chatbot with OpenAI

Let’s make a chatbot! We’ll be using the OpenAI API and Gradio, a Python tool for quickly creating a web interface for machine learning models. Don’t worry if you’ve never used Gradio before (or even Python). ChatGPT will generate some code, and then I walk you through the process of running it. (Or you can try asking ChatGPT how to run it.)

Submit the following prompt (or something similar) to ChatGPT or the OpenAI Playground:

```
Write a custom chatbot that uses the OpenAI API and
Gradio. The chatbot should first ask the user what kind
of bot they'd like to talk to (such as 'coding assistant'
or 'therapist') and then present them with input and
output areas.
```

Note that this prompt is vague. With the first prompt, my goal is just to get something that works. After I see what ChatGPT generates, I can refine my prompt or start working with the code myself.

Listing 3-1 shows the Python code that ChatGPT returned.

LISTING 3-1 A GPT-Generated Chatbot

```
import openai
import gradio as gr
# Set your OpenAI API key
openai.api_key = "YOUR_OPENAI_API_KEY"
def chatbot_response(input_text):
    response = openai.Completion.create(
        engine="davinci",
        prompt=f"I'd like to talk to a {input_text}.",
        max_tokens=50  # You can adjust this for longer
responses
    )
    return response.choices[0].text
# Create the Gradio interface
iface = gr.Interface(
    fn=chatbot_response,
    inputs=gr.inputs.Textbox("text", label="Enter the type
of bot you want to talk to (e.g., 'coding assistant' or
'therapist')"),
    outputs="text",
    live=True,
    title="Custom Chatbot",
    description="Select the type of bot you'd like to talk
to and start the conversation."
```

```
)
iface.launch()
```

At this point, I don't yet know whether the code will work, but I can see that it's not what I meant. But, I see a problem I'd like to fix right away: The API key is hard-coded in the file, which would result in me checking my API key into my Git repository, which I just said you should never do.

I could fix this problem myself, but I decided to let ChatGPT do it. Here's my next prompt:

```
I don't want to hard code my API key into this file.
How can I avoid doing that?
```

In response to this prompt, ChatGPT suggested setting an environment variable containing the API key, which wasn't what I was looking for. I responded that I wanted to have a config file for the API key, and it gave me what I was looking for and correctly instructed me to add the config file to .gitignore. Listing 3-2 shows the revised code.

LISTING 3-2 My revised Chatbot

```
import openai
import gradio as gr
from config import OPENAI_API_KEY
# Set the API key from the configuration file
openai.api_key = OPENAI_API_KEY
def chatbot_response(input_text):
    response = openai.Completion.create(
        engine="davinci",
        prompt=f"I'd like to talk to a {input_text}.",
        max_tokens=50  # You can adjust this for longer
responses
    )
    return response.choices[0].text
# Create the Gradio interface
iface = gr.Interface(
    fn=chatbot_response,
```

```
    inputs=gr.inputs.Textbox("text", label="Enter the type
of bot you want to talk to (e.g., 'coding assistant' or
'therapist')"),
    outputs="text",
    live=True,
    title="Custom Chatbot",
    description="Select the type of bot you'd like to talk
to and start the conversation."
)
iface.launch()
```

To test this script, copy the code into a file named chatbot_demo.py and open it in VS Code. At this point, VS Code may prompt you to install a Python interpreter if you don't already have one, or you may need to install Python on your computer (which you can do by going to `https://www.python.org/downloads/`).

Create a second file named config.py for your API key. The config.py file should look like Listing 3-3, with your API key inserted at the appropriate place, of course.

LISTING 3-3 The config.py File

```
OPENAI_API_KEY = "YOUR_API_KEY_HERE"
```

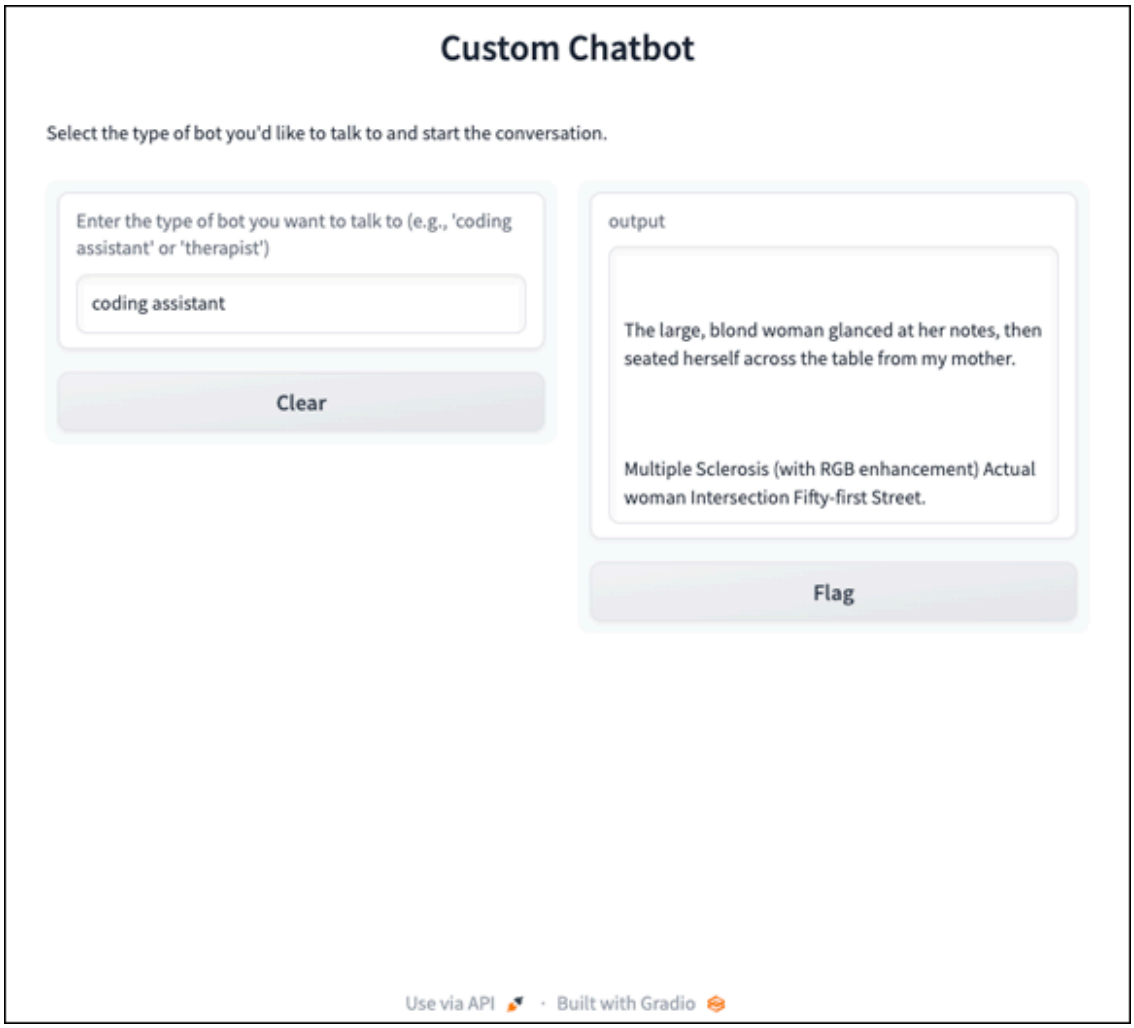
Before you can run the chatbot, you'll need to install Gradio and the OpenAI library. Run the following two commands in your terminal:

```
pip install openai
pip install gradio
```

When those are installed, you can run the program by typing **python chatbot_demo.py** into your terminal.

This code technically works in that it sends the text I enter into the textbox to the OpenAI API and displays a result. But it's not what I had in mind, and the completion it returns is gibberish, as shown in Figure 3-22.

The biggest issue is that the program is sending every keystroke to the API, rather than waiting for me to finish my input and click a button. Checking my OpenAI platform account, I see that my quick test of this program cost me 3 cents. It's not much, but I suspect changing the code so it waits to send prompts all at once rather than sending one letter at a time would save me a lot of money and result in better output.



ChatGPT

FIGURE 3-22: My first attempt at creating a chatbot with ChatGPT was a failure.

Simply changing the value of the `live` property from `True` to `False` solves this problem.

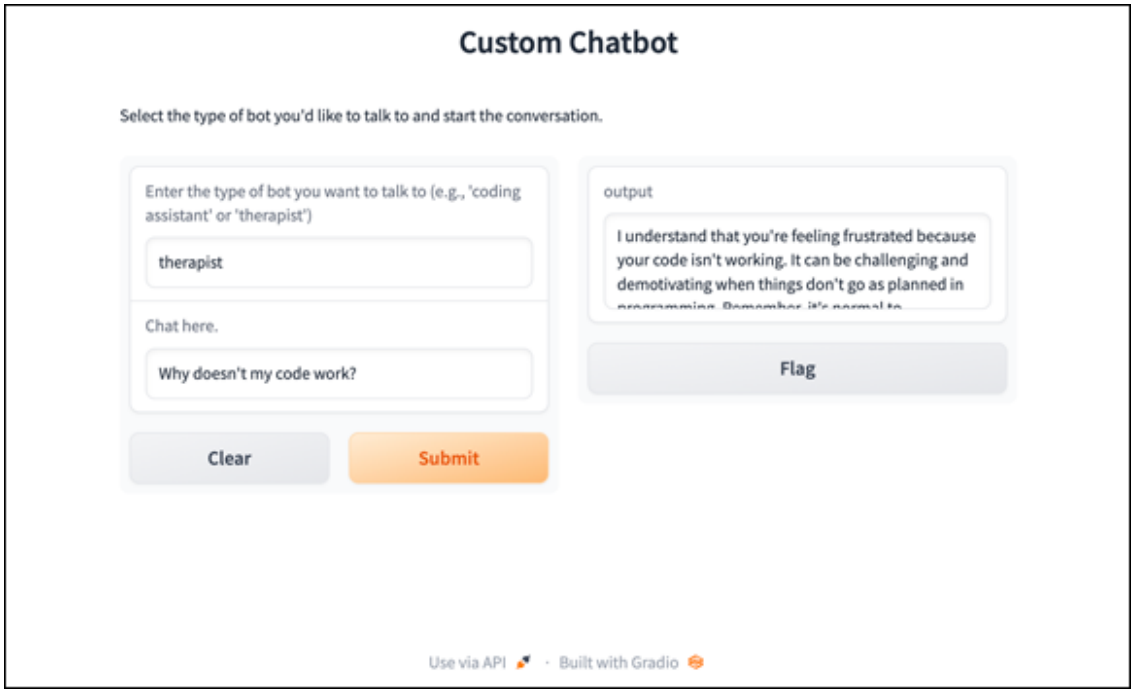
The next problem is that the model is using the GPT-3 model. I suspect the results I get will be much better if I switch to using a newer model. To fix this, I changed `model="davinci"` to `model="gpt-3.5-turbo"` and then stopped and restarted the Python program. This time, when I tried to

submit my input, I got an error in the console that the endpoint didn't support chat completions.

I consulted the Python OpenAI docs and found that I needed to change `openai.Completion` to `openai.ChatCompletion` and change the parameters I was passing to the OpenAI. I tested it again, and the result is shown in Figure 3-23.

The process I went through to get a working demo involved browsing the docs in the Python OpenAI library and the Gradio library more than getting helpful tips from AI models. In the end, although I got my chatbot working, I was left feeling like it would have been much faster to code the app from scratch without the help of AI.

In the next chapter of this book, I show you a better approach to working with generative AI that is more likely to result in working code that does what it's supposed to do.



ChatGPT

FIGURE 3-23: Testing my fixes.